CS790 Assignment 3 Task 2

Title: CS790 Assignment 3 Task 2

Name: Vansil Rakeshkumar Chauhan `24m0847`

Assumptions/Task-Understanding

1. You need to understand the Tor relay selection algorithm. For this you can do two things - refer to the Tor path spec file (shared as a reading reference) and/or inspect the source code (`circ.c` file in Tor source code).
    - **Assumption/Understanding:** It is asking to learn how Tor relay selection algorithm fetches relay data from consensus plain text file.
2. Thereafter, you would need to design your own relay selection simulator. For this, you can pre-fetch the router consensus file directly from the Tor metrics site (or by doing a directory fetch).
    - **Assumption/Understanding:** Create relay selection simulator which fetches the latest consensus file from Tor metrics site, creates list of relays with it's properties (fingerprint, nickname, flag, bandwidth, etc...).
3. Thus, you should be able to generate random circuits from the consensus file.
    - **Assumption/Understanding:** From the list of relays, algorithm should select appropriate relays for `Guard`, `Middle` and `Exit` nodes for a circuit and generate some random working circuit.

4. You can use that with your exercise 3 to construct custom Tor circuit afterward.
    - **Assumption/Understanding:** Use current custom algorithm to create 4 or more node circuit for Task 1.3.

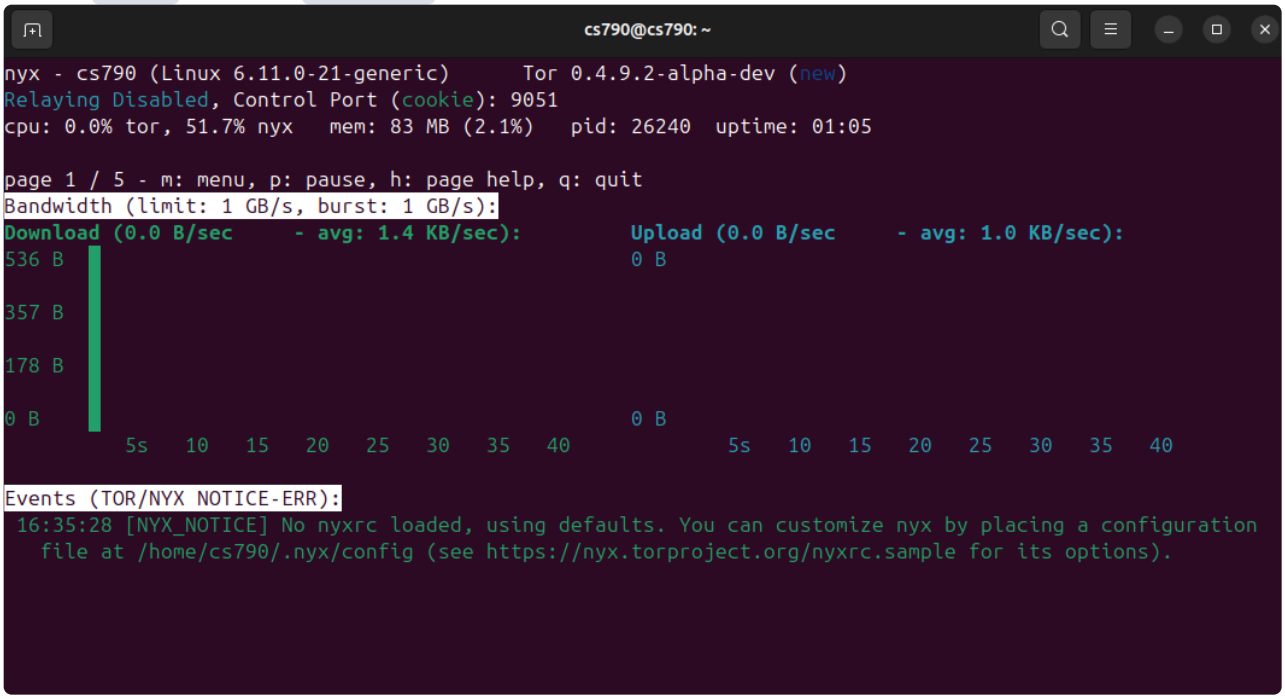
Grading Scheme (35 points)

0. Run Tor and Script

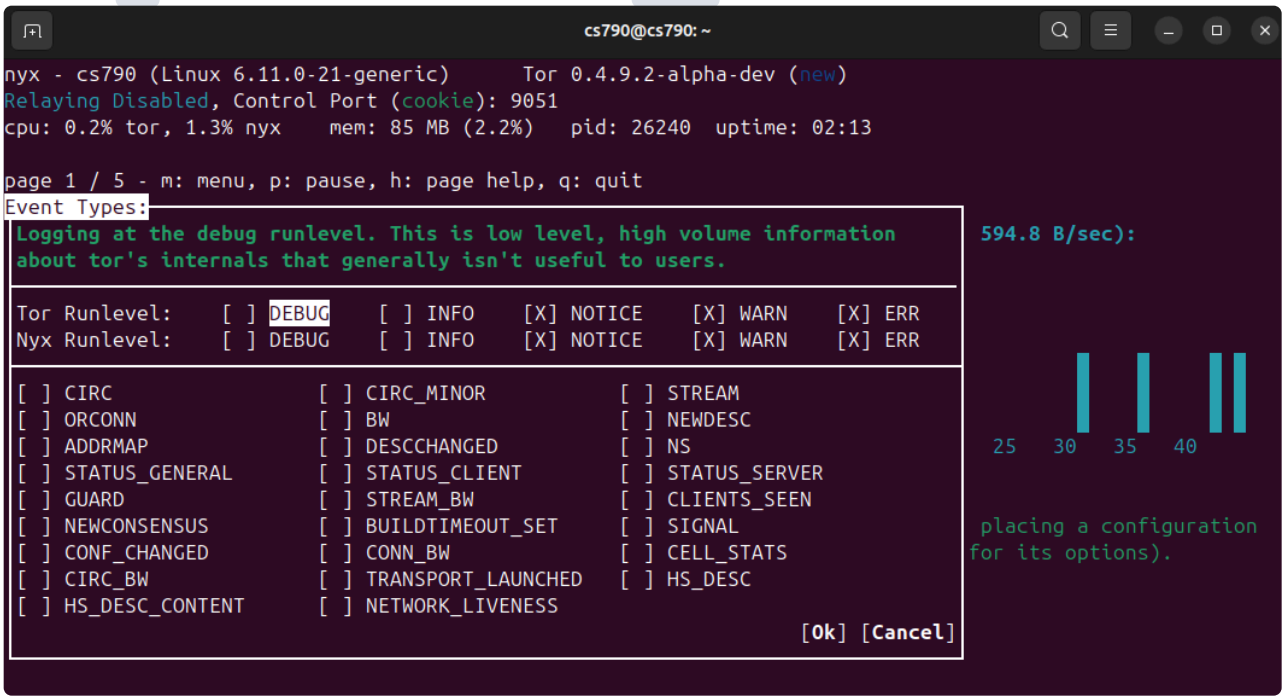
- Run Tor using `$ tor -f /usr/local/etc/tor/torrc`

```
cs790@cs790:~$ tor -f /usr/local/etc/tor/torrc
Apr 15 16:34:24.715 [notice] Tor 0.4.9.2-alpha-dev (git-565390ee632e153a) running o
n Linux with Libevent 2.1.12-stable, OpenSSL 3.0.13, Zlib 1.3, Liblzma N/A, Libzstd
 N/A and Glibc 2.39 as libc.
Apr 15 16:34:24.715 [notice] Tor can't help you if you use it wrong! Learn how to b
e safe at https://support.torproject.org/faq/staying-anonymous/
Apr 15 16:34:24.715 [notice] This version is not a stable Tor release. Expect more
bugs than usual.
Apr 15 16:34:24.716 [notice] Read configuration file "/usr/local/etc/tor/torrc".
Apr 15 16:34:24.718 [notice] Opening Socks listener on 127.0.0.1:9050
Apr 15 16:34:24.718 [notice] Opened Socks listener connection (ready) on 127.0.0.1:
9050
Apr 15 16:34:24.718 [notice] Opening Control listener on 127.0.0.1:9051
Apr 15 16:34:24.718 [notice] Opened Control listener connection (ready) on 127.0.0.
1:9051
Apr 15 16:34:24.000 [notice] Parsing GEOIP IPv4 file /usr/local/share/tor/geoip.
Apr 15 16:34:24.000 [notice] Parsing GEOIP IPv6 file /usr/local/share/tor/geoip6.
Apr 15 16:34:24.000 [notice] Bootstrapped 0% (starting): Starting
Apr 15 16:34:25.000 [notice] Starting with guard context "default"
Apr 15 16:34:26.000 [notice] Bootstrapped 5% (conn): Connecting to a relay
Apr 15 16:34:26.000 [notice] Bootstrapped 10% (conn_done): Connected to a relay
Apr 15 16:34:26.000 [notice] Bootstrapped 14% (handshake): Handshaking with a relay
Apr 15 16:34:26.000 [notice] Bootstrapped 15% (handshake_done): Handshake with a re
lay done
Apr 15 16:34:26.000 [notice] Bootstrapped 75% (enough_dirinfo): Loaded enough direc
tory info to build circuits
Apr 15 16:34:26.000 [notice] Bootstrapped 90% (ap_handshake_done): Handshake finish
ed with a relay to build circuits
Apr 15 16:34:26.000 [notice] Bootstrapped 95% (circuit_create): Establishing a Tor
circuit
Apr 15 16:34:27.000 [notice] Bootstrapped 100% (done): Done
```

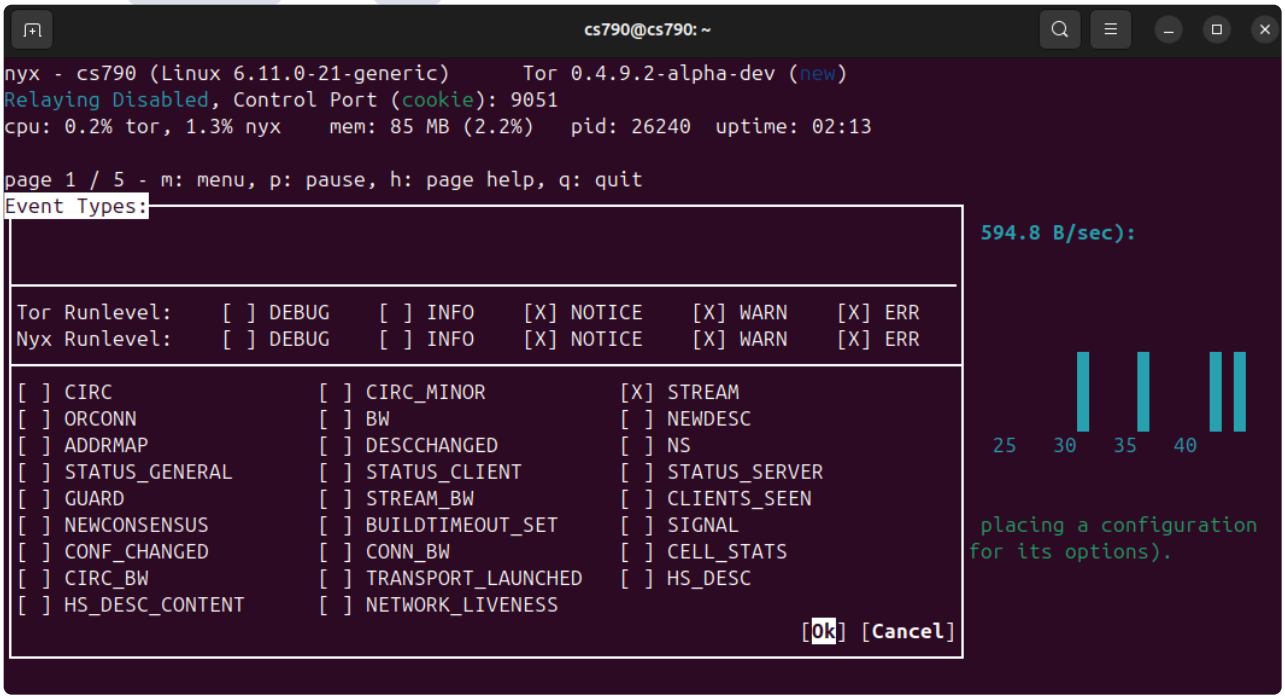
- Run `nyx` using `$ nyx`



- Enable `STREAM` event logs
- Press `E` while on main screen in `nyx` window

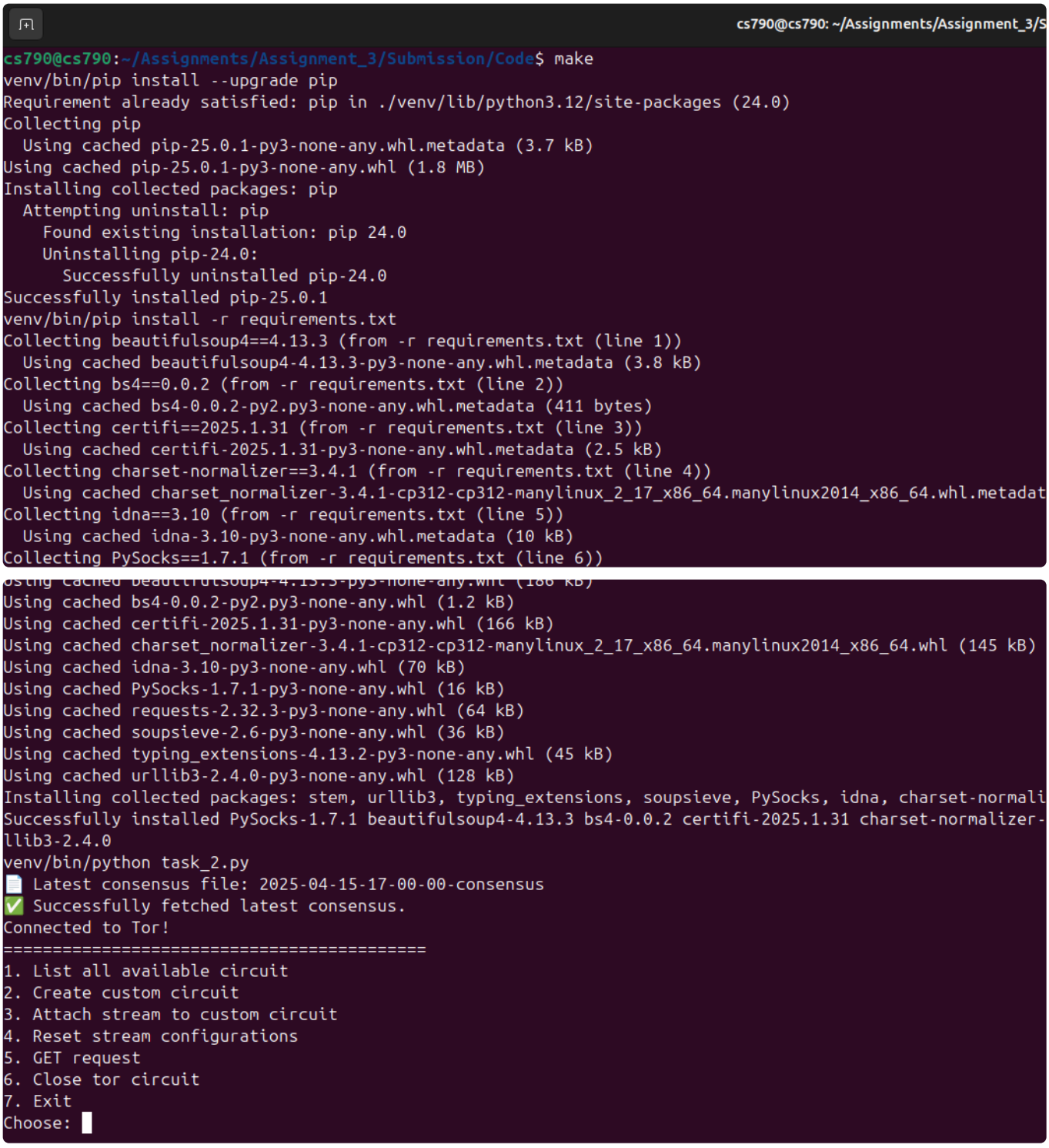


- Select `STREAM` and `OK`



- Go to code directory and Run `MakeFile` using following steps

```
make clean
make
```



---

1. Circuit/patch simulation and verification by creating circuits as suggested by the simulation. (20 points)

Fetching and parsing consensus file logic

```
# Downloads the most recent Tor relay consensus file from the Tor Project's collector
def fetch_latest_consensus():
    # Base URL where recent consensus files are hosted
    base_url = "https://collector.torproject.org/recent/relay-descriptors/consensuses/"

    # Request the directory listing from the Tor metrics collector
    response = requests.get(base_url)
    if response.status_code != 200:
        raise Exception(f"Failed to list consensus files: {response.status_code}")

    # Parse the HTML response to extract links to consensus files
    soup = BeautifulSoup(response.text, 'html.parser')
    links = [a['href'] for a in soup.find_all('a', href=True) if 'consensus' in a['href']]
    links = sorted(links, reverse=True)  # Sort to get the most recent file first

    # Get the most recent consensus file
    latest_file = links[0]
    print(f"📄 Latest consensus file: {latest_file}")

    # Build the full URL to the consensus file and fetch it
    consensus_url = base_url + latest_file
    consensus_resp = requests.get(consensus_url)
    if consensus_resp.status_code != 200:
        raise Exception(f"Failed to fetch consensus: {consensus_resp.status_code}")
```

```
    print("✅ Successfully fetched latest consensus.")

    # Save the consensus file to disk as 'consensus.txt'
    with open("consensus.txt", "w", encoding='utf-8') as
f:
        f.write(consensus_resp.text)

# Relay class to store parsed information about Tor relays
class Relay:
    def __init__(self, fingerprint, nickname, flags,
bandwidth, address, or_port):
        self.fingerprint = fingerprint
        self.nickname = nickname
        self.flags = set(flags)
        self.bandwidth = bandwidth
        self.address = address
        self.or_port = or_port

    def __repr__(self):
        return f"<Relay {self.nickname}
({self.fingerprint[:6]}...) {self.flags} BW=
{self.bandwidth} IP={self.address}:{self.or_port}>"

# Parses a Tor consensus file to extract relay information
def parse_consensus_file(path):
    relays = []
    with open(path, 'r') as f:
        content = f.read()

    # Break the file into relay sections, starting with 'r
' lines
    relay_sections = re.split(r'\nr ', '\n' + content)

    for section in relay_sections[1:]:
        lines = section.splitlines()
```

```
        try:
            # Parse the 'r' line to extract identity
information
            r_parts = lines[0].split()
            nickname = r_parts[0]
            fingerprint_b64 = r_parts[1]
            address = r_parts[5]
            or_port = int(r_parts[6])

            # Convert fingerprint from base64 to hex
format
            fingerprint_hex =
base64.b64decode(fingerprint_b64 + '==').hex().upper()

            # Extract relay flags from the 's' line
            s_line = next((l for l in lines if
l.startswith('s ')), '')
            flags = s_line.split()[1:] if s_line else []

            # Extract bandwidth information from the 'w'
line
            w_line = next((l for l in lines if
l.startswith('w ')), '')
            bandwidth = 0
            if w_line:
                bw_match = re.search(r'Bandwidth=(\d+)',
w_line)
                if bw_match:
                    bandwidth = int(bw_match.group(1))

            # Create Relay object and add to the list
            relay = Relay(fingerprint_hex, nickname,
flags, bandwidth, address, or_port)
            relays.append(relay)
```

```
        except Exception as e:
            print(f"Failed to parse relay section: {e}")
            continue

    return relays
```

Circuit creation logic code

```
# Helper function to select a relay based on bandwidth-
weighted probability
def weighted_choice(relays):
    total_bw = sum(r.bandwidth for r in relays if
r.bandwidth > 0)
    if total_bw == 0:
        return random.choice(relays)
    r = random.uniform(0, total_bw)
    upto = 0
    for relay in relays:
        if relay.bandwidth <= 0:
            continue
        upto += relay.bandwidth
        if upto >= r:
            return relay
    return random.choice(relays)

# Guard relay selection logic
def select_guard(relays):
    eligible = [r for r in relays if 'Guard' in r.flags
and 'Running' in r.flags and 'Valid' in r.flags]
    return weighted_choice(eligible)

# Middle relay selection logic (excluding certain nodes)
def select_middle(relays, exclude):
    eligible = [r for r in relays if 'Running' in r.flags
```

```
and 'Valid' in r.flags and r not in exclude]
    return weighted_choice(eligible)

# Exit relay selection logic
def select_exit(relays, exclude):
    eligible = [r for r in relays if 'Exit' in r.flags and
'Running' in r.flags and 'Valid' in r.flags and r not in
exclude]
    return weighted_choice(eligible)

# Creates a custom n-hop circuit using given relays
def create_n_hop_circuit(controller, relays, hop_count):
    circuit_id = -1
    for attempt in range(15):
        print(f"Attempt {attempt+1}/15")

        try:
            guard = select_guard(relays)
            exit_node = select_exit(relays, exclude=
{guard})

            middle_nodes = []
            used_nodes = {guard, exit_node}

            # Select unique middle nodes
            for _ in range(hop_count - 2):
                node = select_middle(relays,
exclude=used_nodes)
                if node is None:
                    raise Exception("Unable to find unique
middle node")
                middle_nodes.append(node)
                used_nodes.add(node)

            # Build circuit using selected relays
```

```
            circuit = [guard.fingerprint] + [n.fingerprint
for n in middle_nodes] + [exit_node.fingerprint]
            circuit_id =
controller.new_circuit(path=circuit, await_build=True)
            break

        except Exception as e:
            print(f"Failed to build circuit: {e}")
    return int(circuit_id)
```

Basic Functionalities

- `List all available circuit`

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 22 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 126 (hops: 3)
5. Circuit id: 23 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 128 (hops: 3)
8. Circuit id: 127 (hops: 3)
--------------------
```

- `Create custom circuit`

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 2
-------------------------
Specify required hop count: 4
It may take a while. If it takes too long (~15-20s)... please retry.
Attempt 1/15
Circuit id: 129
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 22 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 126 (hops: 3)
5. Circuit id: 23 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 128 (hops: 3)
8. Circuit id: 127 (hops: 3)
9. Circuit id: 129 (hops: 4)
--------------------
```

- `Attach stream to custom circuit`

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 3
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 22 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 126 (hops: 3)
5. Circuit id: 23 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 128 (hops: 3)
8. Circuit id: 127 (hops: 3)
9. Circuit id: 129 (hops: 4)
--------------------
Enter Circuit ID: 129
Attached listener to EventType.STREAM
------------------------------------------
```

- `Get request through tor proxy`

```
--------------------------
-------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 15 (hops: 4)
-------------------
Enter Circuit ID: 15
Attached listener to EventType.STREAM
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 5
--------------------------
Enter url: https://facebook.com
Status Code: 200
Response Snippet:
<!DOCTYPE html>
<html lang="de" id="facebook" class="no_js">
<head><meta charset="utf-8" /><meta name="referrer" content="default"
Lazy?window.requireLazy(["Env"],b):(window.Env=window.Env||{},b(window
trace_limit":30,"timesliceBufferSize":5000,"show_invariant_
==========================================
```

- `Remove stream configurations`

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 4
-------------------------
Success: Removed event listener
Success: Reset config
==========================================
```

- `Close tor circuit`

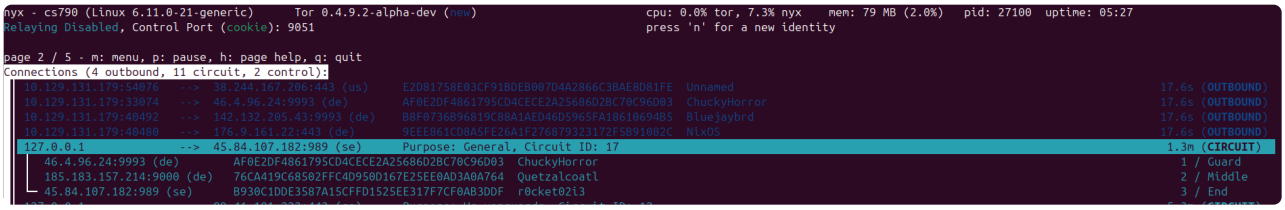
```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 6
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 15 (hops: 4)
--------------------
Enter Circuit ID: 15
Closed circuit 15
```

Simulating circuit creation

- Creating `3` hop custom circuit

```
📄 Latest consensus file: 2025-04-15-17-00-00-consensus
✅ Successfully fetched latest consensus.
Connected to Tor!
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 2
--------------------------
Specify required hop count: 3
It may take a while. If it takes too long (~15-20s)... please retry.
Attempt 1/15
Failed to build circuit: Circuit failed to be created: CHANNEL_CLOSED
Attempt 2/15
Circuit id: 17
==========================================
```

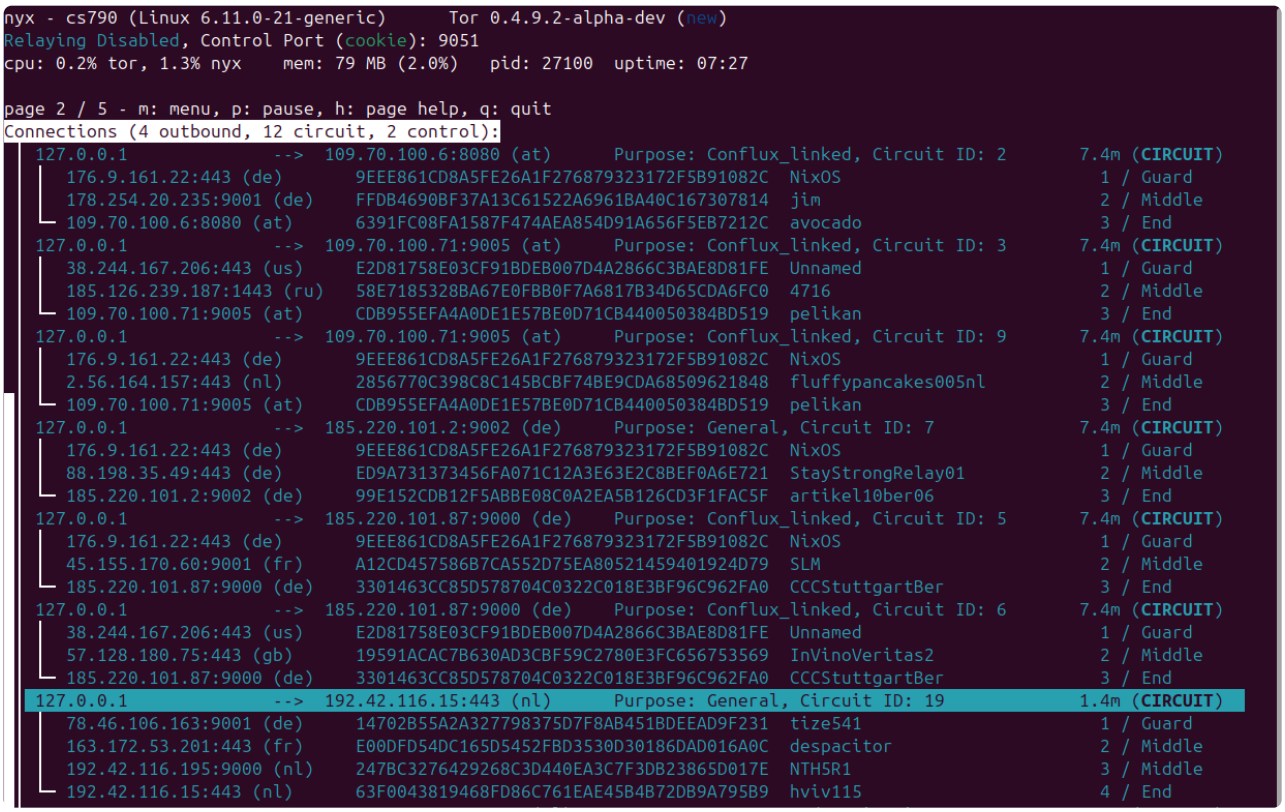
```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
--------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
--------------------
```



- Creating `4` hop custom circuit

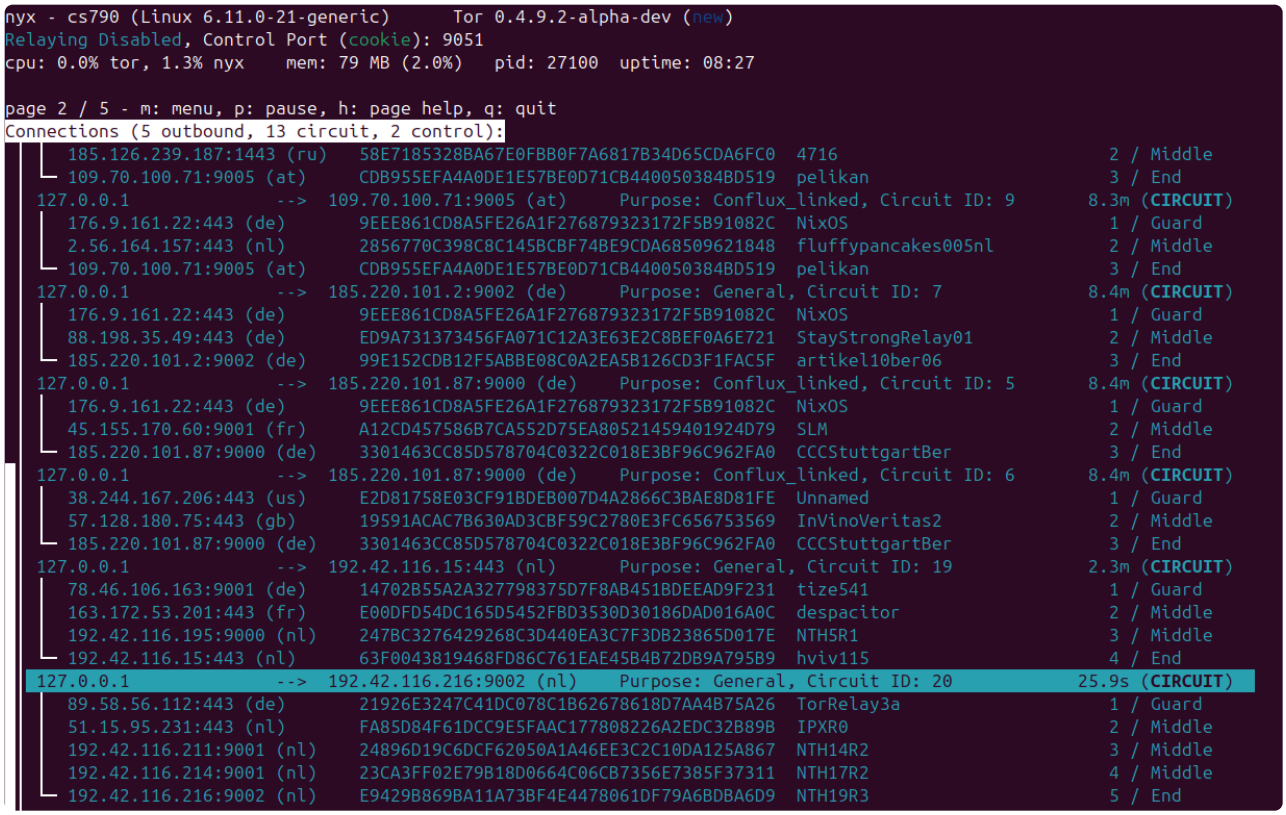
```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 2
-------------------------
Specify required hop count: 4
It may take a while. If it takes too long (~15-20s)... please retry.
Attempt 1/15
Failed to build circuit: Circuit failed to be created: CHANNEL_CLOSED
Attempt 2/15
Circuit id: 19
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
12. Circuit id: 19 (hops: 4)
--------------------
```

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 2
-------------------------
Specify required hop count: 4
It may take a while. If it takes too long (~15-20s)... please retry.
Attempt 1/15
Failed to build circuit: Circuit failed to be created: CHANNEL_CLOSED
Attempt 2/15
Circuit id: 19
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
12. Circuit id: 19 (hops: 4)
--------------------
```


nyx - cs790 (Linux 6.11.0-21-generic) Tor 0.4.9.2-alpha-dev (new)
Relaying Disabled, Control Port (cookie): 9051
cpu: 0.2% tor, 1.3% nyx mem: 79 MB (2.0%) pid: 27100 uptime: 07:27
page 2 / 5 - m: menu, p: pause, h: page help, q: quit
Connections (4 outbound, 12 circuit, 2 control):
127.0.0.1 --> 109.70.100.6:8080 (at) Purpose: Conflux_linked, Circuit ID: 2 7.4m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
178.254.20.235:9001 (de) FFDB4690BF37A13C61522A6961BA40C167307814 jim 2 / Middle
109.70.100.6:8080 (at) 6391FC08FA1587F474AEA854D91A656F5EB7212C avocado 3 / End
127.0.0.1 --> 109.70.100.71:9005 (at) Purpose: Conflux_linked, Circuit ID: 3 7.4m (CIRCUIT)
38.244.167.206:443 (us) E2D81758E03CF91BDEB007D4A2866C3BAE8D81FE Unnamed 1 / Guard
185.126.239.187:1443 (ru) 58E7185328BA67E0FBB0F7A6817B34D65CDA6FC0 4716 2 / Middle
109.70.100.71:9005 (at) CDB955EFA4A0DE1E57BE0D71CB440050384BD519 pelikan 3 / End
127.0.0.1 --> 109.70.100.71:9005 (at) Purpose: Conflux_linked, Circuit ID: 9 7.4m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
2.56.164.157:443 (nl) 2856770C398C8C145BCBF74BE9CDA68509621848 fluffypancakes005nl 2 / Middle
109.70.100.71:9005 (at) CDB955EFA4A0DE1E57BE0D71CB440050384BD519 pelikan 3 / End
127.0.0.1 --> 185.220.101.2:9002 (de) Purpose: General, Circuit ID: 7 7.4m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
88.198.35.49:443 (de) ED9A731373456FA071C12A3E63E2C8BEF0A6E721 StayStrongRelay01 2 / Middle
185.220.101.2:9002 (de) 99E152CDB12F5ABBE08C0A2EA5B126CD3F1FAC5F artikel10ber06 3 / End
127.0.0.1 --> 185.220.101.87:9000 (de) Purpose: Conflux_linked, Circuit ID: 5 7.4m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
45.155.170.60:9001 (fr) A12CD457586B7CA552D75EA80521459401924D79 SLM 2 / Middle
185.220.101.87:9000 (de) 3301463CC85D578704C0322C018E3BF96C962FA0 CCCStuttgartBer 3 / End
127.0.0.1 --> 185.220.101.87:9000 (de) Purpose: Conflux_linked, Circuit ID: 6 7.4m (CIRCUIT)
38.244.167.206:443 (us) E2D81758E03CF91BDEB007D4A2866C3BAE8D81FE Unnamed 1 / Guard
57.128.180.75:443 (gb) 19591ACAC7B630AD3CBF59C2780E3FC656753569 InVinoVeritas2 2 / Middle
185.220.101.87:9000 (de) 3301463CC85D578704C0322C018E3BF96C962FA0 CCCStuttgartBer 3 / End
127.0.0.1 --> 192.42.116.15:443 (nl) Purpose: General, Circuit ID: 19 1.4m (CIRCUIT)
78.46.106.163:9001 (de) 14702B55A2A327798375D7F8AB451BDEEAD9F231 tize541 1 / Guard
163.172.53.201:443 (fr) E00DFD54DC165D5452FBD3530D30186DAD016A0C despacitor 2 / Middle
192.42.116.195:9000 (nl) 247BC3276429268C3D440EA3C7F3DB23865D017E NTH5R1 3 / Middle
192.42.116.15:443 (nl) 63F0043819468FD86C761EAE45B4B72DB9A795B9 hviv115 4 / End

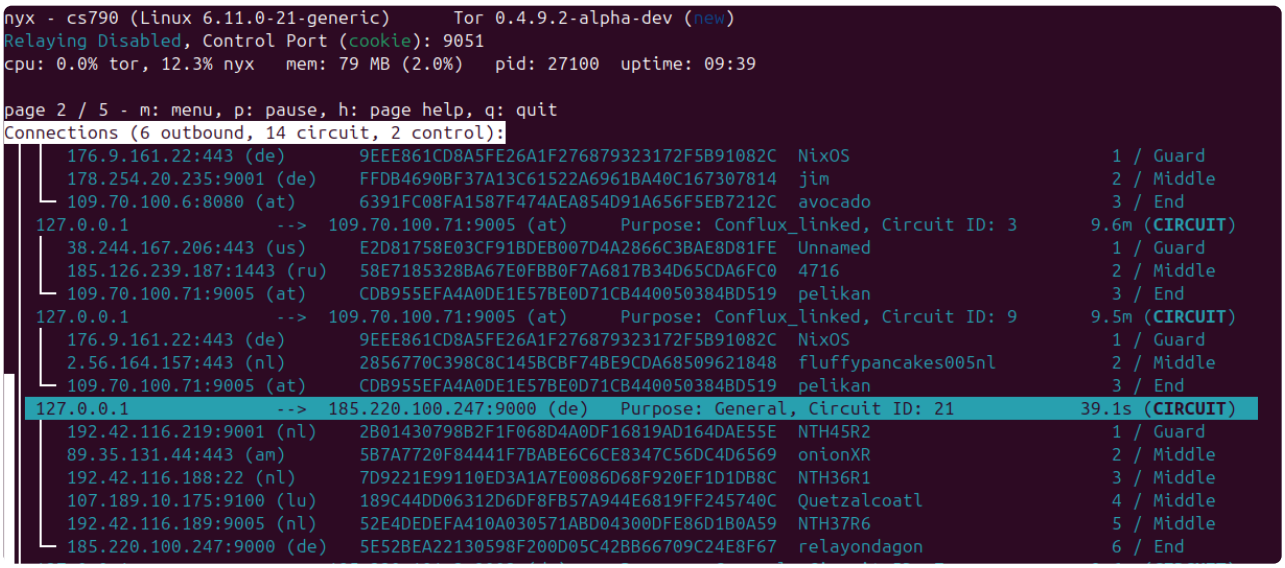
- Creating `5` hop custom circuit

```
------------------
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 2
-------------------------
Specify required hop count: 5
It may take a while. If it takes too long (~15-20s)... please retry.
Attempt 1/15
Circuit id: 20
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
-------------------------
------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
12. Circuit id: 19 (hops: 4)
13. Circuit id: 20 (hops: 5)
------------------
```


nyx - cs790 (Linux 6.11.0-21-generic) Tor 0.4.9.2-alpha-dev (new)
Relaying Disabled, Control Port (cookie): 9051
cpu: 0.0% tor, 1.3% nyx mem: 79 MB (2.0%) pid: 27100 uptime: 08:27
page 2 / 5 - m: menu, p: pause, h: page help, q: quit
Connections (5 outbound, 13 circuit, 2 control):
185.126.239.187:1443 (ru) 58E7185328BA67E0FBB0F7A6817B34D65CDA6FC0 4716 2 / Middle
109.70.100.71:9005 (at) CDB955EFA4A0DE1E57BE0D71CB440050384BD519 pelikan 3 / End
127.0.0.1 --> 109.70.100.71:9005 (at) Purpose: Conflux_linked, Circuit ID: 9 8.3m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
2.56.164.157:443 (nl) 2856770C398C8C145BCBF74BE9CDA68509621848 fluffypancakes005nl 2 / Middle
109.70.100.71:9005 (at) CDB955EFA4A0DE1E57BE0D71CB440050384BD519 pelikan 3 / End
127.0.0.1 --> 185.220.101.2:9002 (de) Purpose: General, Circuit ID: 7 8.4m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
88.198.35.49:443 (de) ED9A731373456FA071C12A3E63E2C8BEF0A6E721 StayStrongRelay01 2 / Middle
185.220.101.2:9002 (de) 99E152CDB12F5ABBE08C0A2EA5B126CD3F1FAC5F artikel10ber06 3 / End
127.0.0.1 --> 185.220.101.87:9000 (de) Purpose: Conflux_linked, Circuit ID: 5 8.4m (CIRCUIT)
176.9.161.22:443 (de) 9EEE861CD8A5FE26A1F276879323172F5B91082C NixOS 1 / Guard
45.155.170.60:9001 (fr) A12CD457586B7CA552D75EA805214594019240D79 SLM 2 / Middle
185.220.101.87:9000 (de) 3301463CC85D578704C0322C018E3BF96C962FA0 CCCStuttgartBer 3 / End
127.0.0.1 --> 185.220.101.87:9000 (de) Purpose: Conflux_linked, Circuit ID: 6 8.4m (CIRCUIT)
38.244.167.206:443 (us) E2D81758E03CF91BDEB007D4A2866C3BAE8D81FE Unnamed 1 / Guard
57.128.180.75:443 (gb) 19591ACAC7B630AD3CBF59C2780E3FC656753569 InVinoVeritas2 2 / Middle
185.220.101.87:9000 (de) 3301463CC85D578704C0322C018E3BF96C962FA0 CCCStuttgartBer 3 / End
127.0.0.1 --> 192.42.116.15:443 (nl) Purpose: General, Circuit ID: 19 2.3m (CIRCUIT)
78.46.106.163:9001 (de) 14702B55A2A327798375D7F8AB451BDEEAD9F231 tize541 1 / Guard
163.172.53.201:443 (fr) E00DFD54DC165D5452FBD3530D30186DAD016A0C despacitor 2 / Middle
192.42.116.195:9000 (nl) 247BC3276429268C3D440EA3C7F3DB23865D017E NTH5R1 3 / Middle
192.42.116.15:443 (nl) 63F0043819468FD86C761EAE45B4B72DB9A795B9 hviv115 4 / End
127.0.0.1 --> 192.42.116.216:9002 (nl) Purpose: General, Circuit ID: 20 25.9s (CIRCUIT)
89.58.56.112:443 (de) 21926E3247C41DC078C1B62678618D7AA4B75A26 TorRelay3a 1 / Guard
51.15.95.231:443 (nl) FA85D84F61DCC9E5FAAC177808226A2EDC32B89B IPXR0 2 / Middle
192.42.116.211:9001 (nl) 24896D19C6DCF62050A1A46EE3C2C10DA125A867 NTH14R2 3 / Middle
192.42.116.214:9001 (nl) 23CA3FF02E79B18D0664C06CB7356E7385F37311 NTH17R2 4 / Middle
192.42.116.216:9002 (nl) E9429B869BA11A73BF4E4478061DF79A6BDBA6D9 NTH19R3 5 / End

- Creating `6` hop custom circuit

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 2
-------------------------
Specify required hop count: 6
It may take a while. If it takes too long (~15-20s)... please retry.
Attempt 1/15
Circuit id: 21
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 1
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
12. Circuit id: 19 (hops: 4)
13. Circuit id: 20 (hops: 5)
14. Circuit id: 21 (hops: 6)
--------------------
```



---

2. Circuit is functional and can be used for web-browsing/file-downloading. (10 points)

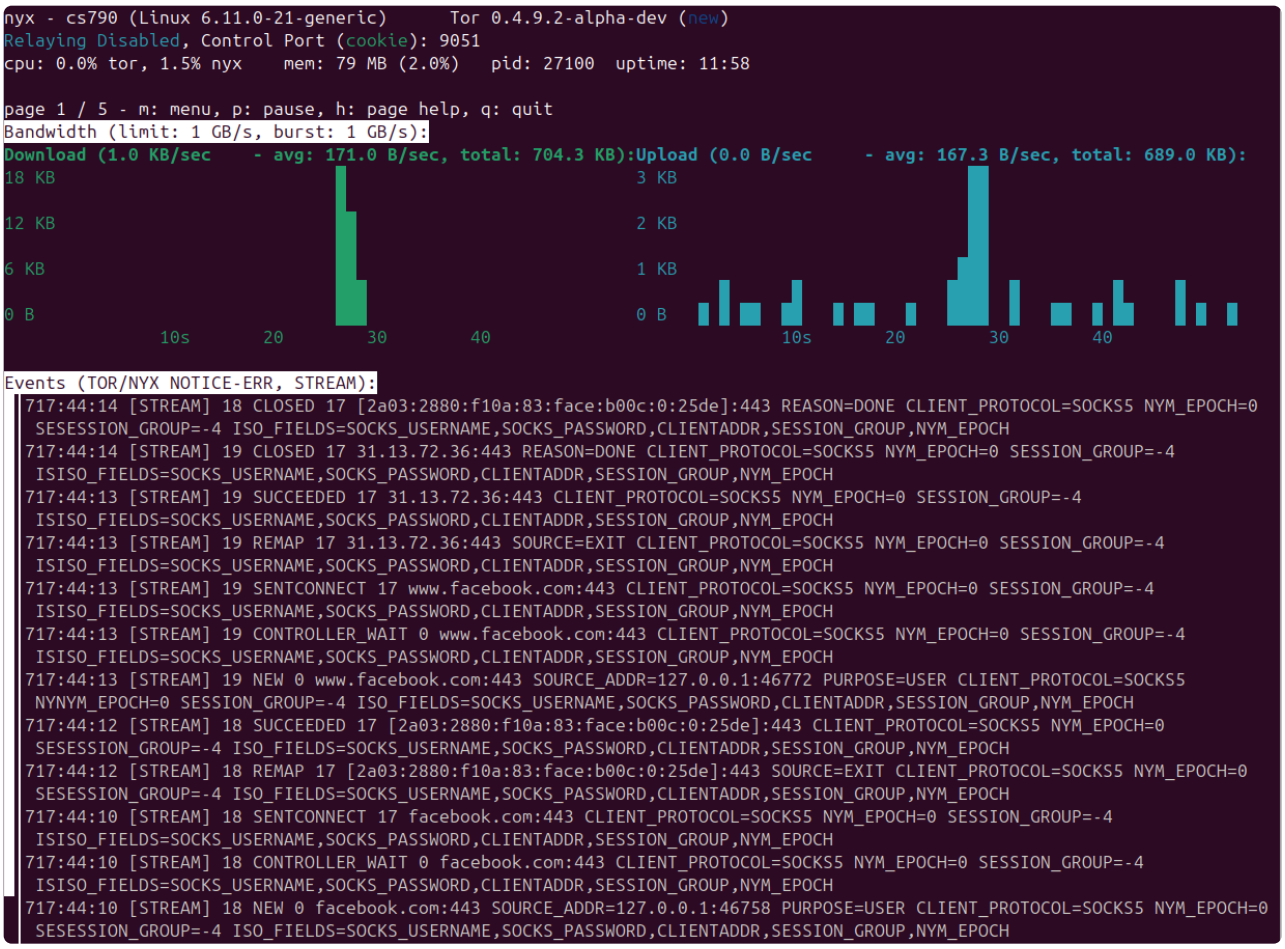
- Attaching stream and Request sending logic code

```
# Attaches new streams to a custom circuit using an event
listener
def attach_stream_to_circuit(controller, circuit_id):
    def attach_stream(stream):
        try:
            if stream.status == 'NEW':
                controller.attach_stream(stream.id,
circuit_id)
        except Exception as e:
            print(f"Failed to attach stream {stream.id}:
{e}")

    controller.add_event_listener(attach_stream,
EventType.STREAM)
    controller.set_conf('__LeaveStreamsUnattached', '1')
    print(f"Attached listener to EventType.STREAM")

# Resets Tor configuration to default behavior
```

```
def reset_tor_config(controller):
    try:
        controller.remove_event_listener(EventType.STREAM)
        print(f"Success: Removed event listener")
    except Exception as e:
        print(f"Warning: could not remove stream listener
- {e}")

    try:
        controller.reset_conf('__LeaveStreamsUnattached')
        print(f"Success: Reset config")
    except Exception as e:
        print(f"Warning: could not reset config - {e}")

# Sends a GET request through the Tor SOCKS proxy
def send_get_request(url):
    proxies = {
        'http': 'socks5h://127.0.0.1:9050',
        'https': 'socks5h://127.0.0.1:9050',
    }

    try:
        response = requests.get(url, proxies=proxies,
timeout=10)
        print(f"Status Code: {response.status_code}")
        print("Response Snippet:")
        print(response.text[:500])  # print only first 500
characters
        return response.text
    except requests.RequestException as e:
        print(f"Request failed: {e}")
        return None
```

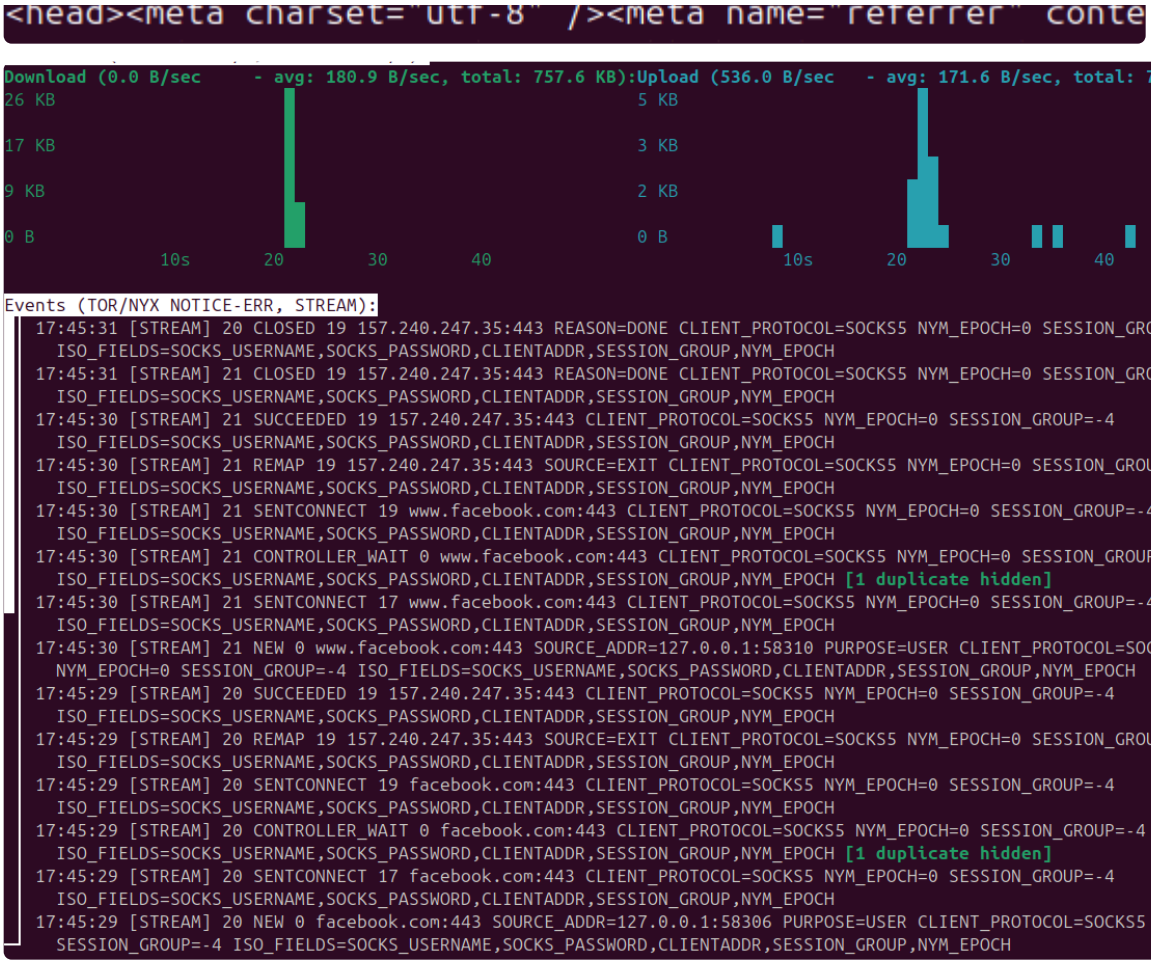
- Selecting and sending GET request on `3` hop custom circuit

```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 3
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
12. Circuit id: 19 (hops: 4)
13. Circuit id: 20 (hops: 5)
14. Circuit id: 21 (hops: 6)
--------------------
Enter Circuit ID: 17
Attached listener to EventType.STREAM
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 5
-------------------------
Enter url: https://facebook.com
Status Code: 200
Response Snippet:
<!DOCTYPE html>
<html lang="sv" id="facebook" class="no_js">
<head><meta charset="utf-8" /><meta name="referrer" content="default" i
```


nyx - cs790 (Linux 6.11.0-21-generic) Tor 0.4.9.2-alpha-dev (new)
Relaying Disabled, Control Port (cookie): 9051
cpu: 0.0% tor, 1.5% nyx mem: 79 MB (2.0%) pid: 27100 uptime: 11:58
page 1 / 5 - m: menu, p: pause, h: page help, q: quit
Bandwidth (limit: 1 GB/s, burst: 1 GB/s):
Download (1.0 KB/sec - avg: 171.0 B/sec, total: 704.3 KB):
Upload (0.0 B/sec - avg: 167.3 B/sec, total: 689.0 KB):
18 KB
12 KB
6 KB
0 B
3 KB
2 KB
1 KB
0 B
10s 20 30 40
10s 20 30 40
Events (TOR/NYX NOTICE-ERR, STREAM):
717:44:14 [STREAM] 18 CLOSED 17 [2a03:2880:f10a:83:face:b00c:0:25de]:443 REASON=DONE CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0
SESESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:14 [STREAM] 19 CLOSED 17 31.13.72.36:443 REASON=DONE CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:13 [STREAM] 19 SUCCEEDED 17 31.13.72.36:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:13 [STREAM] 19 REMAP 17 31.13.72.36:443 SOURCE=EXIT CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:13 [STREAM] 19 SENTCONNECT 17 www.facebook.com:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:13 [STREAM] 19 CONTROLLER_WAIT 0 www.facebook.com:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:13 [STREAM] 19 NEW 0 www.facebook.com:443 SOURCE_ADDR=127.0.0.1:46772 PURPOSE=USER CLIENT_PROTOCOL=SOCKS5
NYNYM_EPOCH=0 SESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:12 [STREAM] 18 SUCCEEDED 17 [2a03:2880:f10a:83:face:b00c:0:25de]:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0
SESESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:12 [STREAM] 18 REMAP 17 [2a03:2880:f10a:83:face:b00c:0:25de]:443 SOURCE=EXIT CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0
SESESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:10 [STREAM] 18 SENTCONNECT 17 facebook.com:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:10 [STREAM] 18 CONTROLLER_WAIT 0 facebook.com:443 CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0 SESSION_GROUP=-4
ISISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH
717:44:10 [STREAM] 18 NEW 0 facebook.com:443 SOURCE_ADDR=127.0.0.1:46758 PURPOSE=USER CLIENT_PROTOCOL=SOCKS5 NYM_EPOCH=0
SESESSION_GROUP=-4 ISO_FIELDS=SOCKS_USERNAME,SOCKS_PASSWORD,CLIENTADDR,SESSION_GROUP,NYM_EPOCH

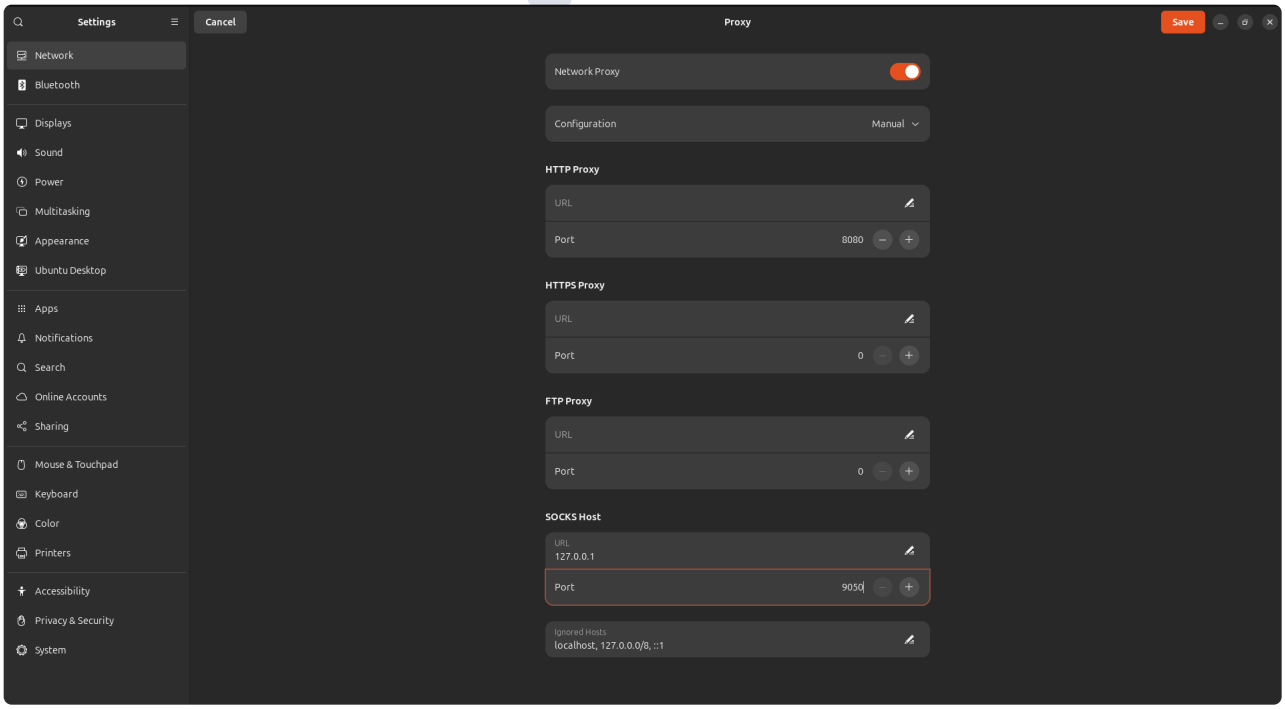
- Sending get request on `4` hop custom circuit

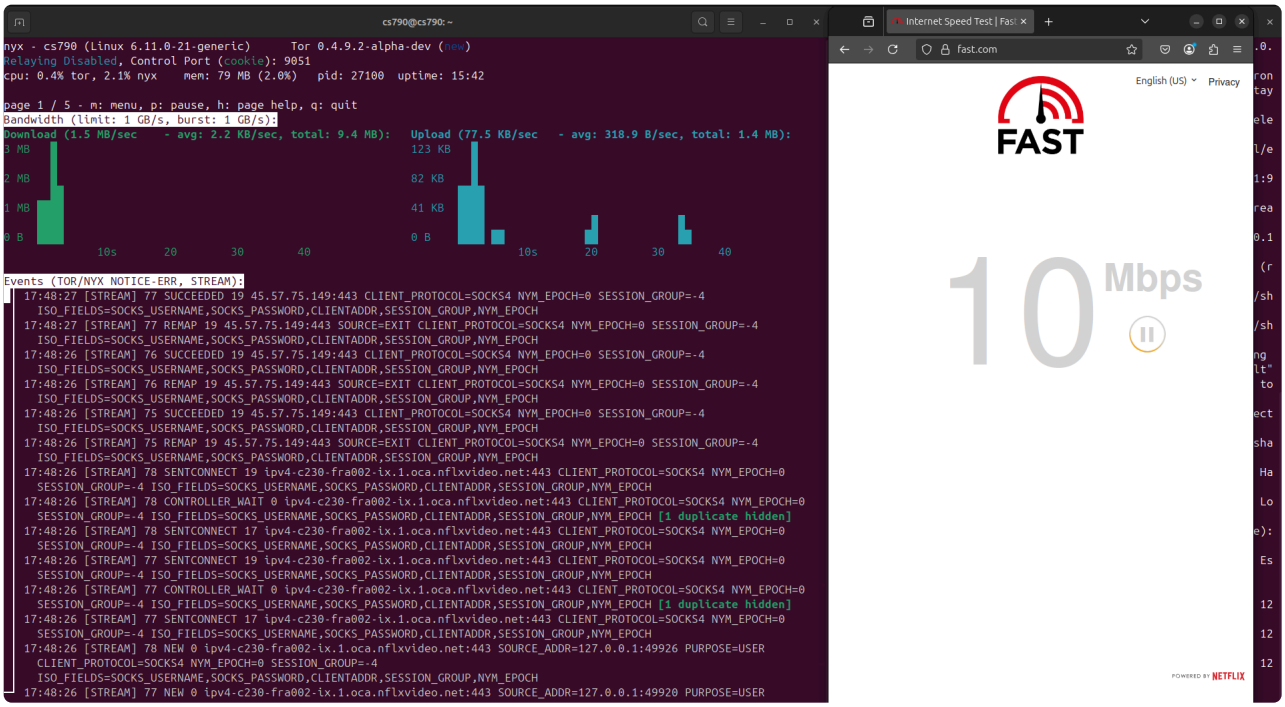
```
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 3
-------------------------
--------------------
Available circuits
1. Circuit id: 1 (hops: 3)
2. Circuit id: 2 (hops: 3)
3. Circuit id: 3 (hops: 3)
4. Circuit id: 9 (hops: 3)
5. Circuit id: 5 (hops: 3)
6. Circuit id: 6 (hops: 3)
7. Circuit id: 7 (hops: 3)
8. Circuit id: 11 (hops: 3)
9. Circuit id: 12 (hops: 3)
10. Circuit id: 13 (hops: 3)
11. Circuit id: 17 (hops: 3)
12. Circuit id: 19 (hops: 4)
13. Circuit id: 20 (hops: 5)
14. Circuit id: 21 (hops: 6)
--------------------
Enter Circuit ID: 19
Attached listener to EventType.STREAM
==========================================
1. List all available circuit
2. Create custom circuit
3. Attach stream to custom circuit
4. Reset stream configurations
5. GET request
6. Close tor circuit
7. Exit
Choose: 5
-------------------------
Enter url: https://facebook.com
Status Code: 200
Response Snippet:
<!DOCTYPE html>
<html lang="nl" id="facebook" class="no_js">
```



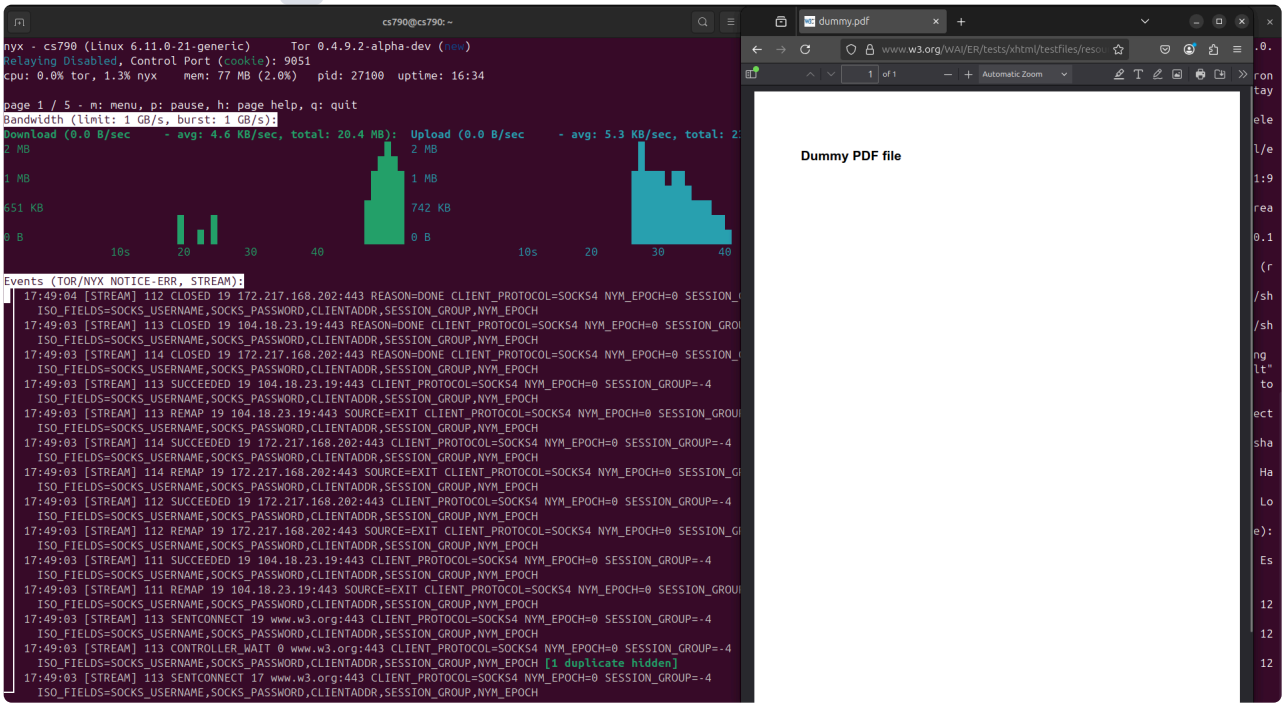
Proxies setup for web-browsing

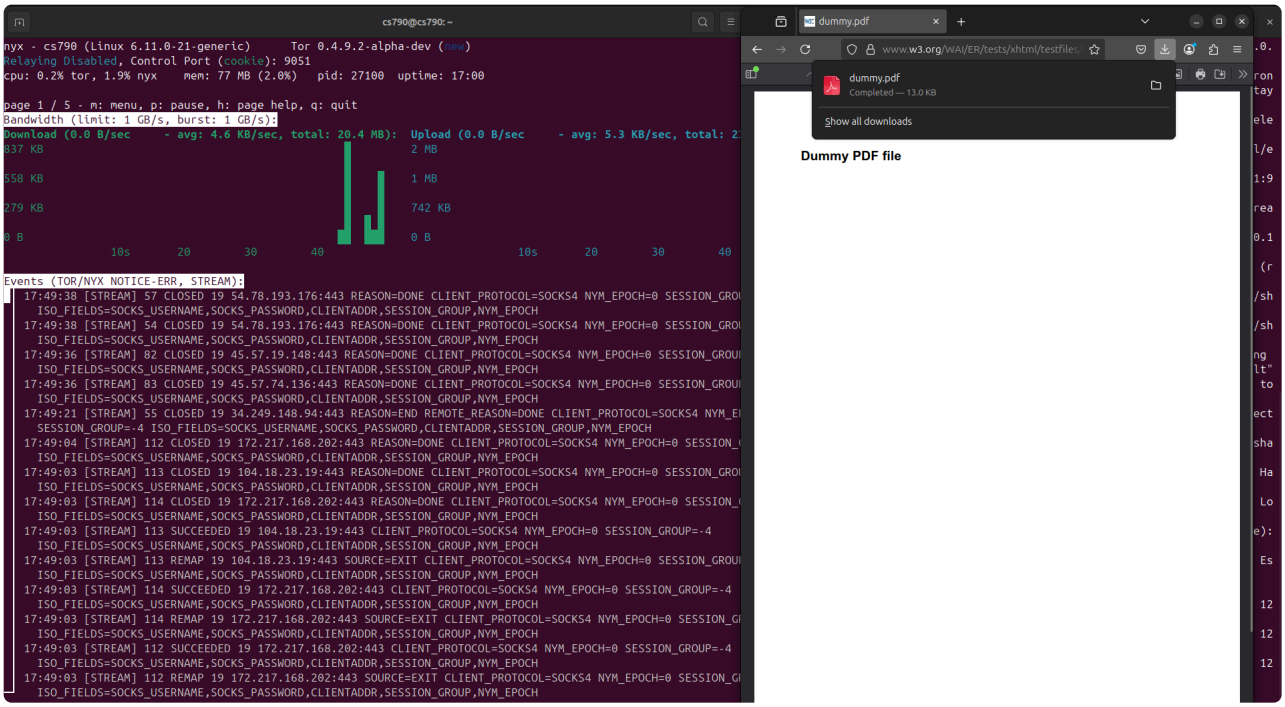
- Setup for web-browsing on `4` hop custom circuit





- Download file `4` hop custom circuit





Submission Files

How the System works

Full Code

```
# Standard libraries
import random
import threading
import re
import base64
import requests

# For scraping latest consensus
from bs4 import BeautifulSoup

# Tor control library
from stem.control import Controller, EventType

# Downloads the most recent Tor relay consensus file from
```

```
the Tor Project's collector
def fetch_latest_consensus():
    # Base URL where recent consensus files are hosted
    base_url =
"https://collector.torproject.org/recent/relay-
descriptors/consensuses/"

    # Request the directory listing from the Tor metrics
collector
    response = requests.get(base_url)
    if response.status_code != 200:
        raise Exception(f"Failed to list consensus files:
{response.status_code}")

    # Parse the HTML response to extract links to
consensus files
    soup = BeautifulSoup(response.text, 'html.parser')
    links = [a['href'] for a in soup.find_all('a',
href=True) if 'consensus' in a['href']]
    links = sorted(links, reverse=True)  # Sort to get the
most recent file first

    # Get the most recent consensus file
    latest_file = links[0]
    print(f"📄 Latest consensus file: {latest_file}")

    # Build the full URL to the consensus file and fetch
it
    consensus_url = base_url + latest_file
    consensus_resp = requests.get(consensus_url)
    if consensus_resp.status_code != 200:
        raise Exception(f"Failed to fetch consensus:
{consensus_resp.status_code}")

    print("✅ Successfully fetched latest consensus.")
```

```
    # Save the consensus file to disk as 'consensus.txt'
    with open("consensus.txt", "w", encoding='utf-8') as
f:
        f.write(consensus_resp.text)


# Relay class to store parsed information about Tor relays
class Relay:
    def __init__(self, fingerprint, nickname, flags,
bandwidth, address, or_port):
        self.fingerprint = fingerprint
        self.nickname = nickname
        self.flags = set(flags)
        self.bandwidth = bandwidth
        self.address = address
        self.or_port = or_port

    def __repr__(self):
        return f"<Relay {self.nickname}
({self.fingerprint[:6]}...) {self.flags} BW=
{self.bandwidth} IP={self.address}:{self.or_port}>"

# Parses a Tor consensus file to extract relay information
def parse_consensus_file(path):
    relays = []
    with open(path, 'r') as f:
        content = f.read()

    # Break the file into relay sections, starting with 'r
' lines
    relay_sections = re.split(r'\nr ', '\n' + content)

    for section in relay_sections[1:]:
        lines = section.splitlines()
```

```
        try:
            # Parse the 'r' line to extract identity
information
            r_parts = lines[0].split()
            nickname = r_parts[0]
            fingerprint_b64 = r_parts[1]
            address = r_parts[5]
            or_port = int(r_parts[6])

            # Convert fingerprint from base64 to hex
format
            fingerprint_hex =
base64.b64decode(fingerprint_b64 + '==').hex().upper()

            # Extract relay flags from the 's' line
            s_line = next((l for l in lines if
l.startswith('s ')), '')
            flags = s_line.split()[1:] if s_line else []

            # Extract bandwidth information from the 'w'
line
            w_line = next((l for l in lines if
l.startswith('w ')), '')
            bandwidth = 0
            if w_line:
                bw_match = re.search(r'Bandwidth=(\d+)',
w_line)
                if bw_match:
                    bandwidth = int(bw_match.group(1))

            # Create Relay object and add to the list
            relay = Relay(fingerprint_hex, nickname,
flags, bandwidth, address, or_port)
            relays.append(relay)
```

```
        except Exception as e:
            print(f"Failed to parse relay section: {e}")
            continue

    return relays

# Helper function to select a relay based on bandwidth-
weighted probability
def weighted_choice(relays):
    total_bw = sum(r.bandwidth for r in relays if
r.bandwidth > 0)
    if total_bw == 0:
        return random.choice(relays)
    r = random.uniform(0, total_bw)
    upto = 0
    for relay in relays:
        if relay.bandwidth <= 0:
            continue
        upto += relay.bandwidth
        if upto >= r:
            return relay
    return random.choice(relays)

# Guard relay selection logic
def select_guard(relays):
    eligible = [r for r in relays if 'Guard' in r.flags
and 'Running' in r.flags and 'Valid' in r.flags]
    return weighted_choice(eligible)

# Middle relay selection logic (excluding certain nodes)
def select_middle(relays, exclude):
    eligible = [r for r in relays if 'Running' in r.flags
and 'Valid' in r.flags and r not in exclude]
    return weighted_choice(eligible)
```

```
# Exit relay selection logic
def select_exit(relays, exclude):
    eligible = [r for r in relays if 'Exit' in r.flags and
'Running' in r.flags and 'Valid' in r.flags and r not in
exclude]
    return weighted_choice(eligible)

# Creates a custom n-hop circuit using given relays
def create_n_hop_circuit(controller, relays, hop_count):
    circuit_id = -1
    for attempt in range(15):
        print(f"Attempt {attempt+1}/15")

        try:
            guard = select_guard(relays)
            exit_node = select_exit(relays, exclude=
{guard})

            middle_nodes = []
            used_nodes = {guard, exit_node}

            # Select unique middle nodes
            for _ in range(hop_count - 2):
                node = select_middle(relays,
exclude=used_nodes)
                if node is None:
                    raise Exception("Unable to find unique
middle node")
                middle_nodes.append(node)
                used_nodes.add(node)

            # Build circuit using selected relays
            circuit = [guard.fingerprint] + [n.fingerprint
for n in middle_nodes] + [exit_node.fingerprint]
            circuit_id =
```

```
controller.new_circuit(path=circuit, await_build=True)
            break

        except Exception as e:
            print(f"Failed to build circuit: {e}")
    return int(circuit_id)

# Attaches new streams to a custom circuit using an event
listener
def attach_stream_to_circuit(controller, circuit_id):
    def attach_stream(stream):
        try:
            if stream.status == 'NEW':
                controller.attach_stream(stream.id,
circuit_id)
        except Exception as e:
            print(f"Failed to attach stream {stream.id}:
{e}")

    controller.add_event_listener(attach_stream,
EventType.STREAM)
    controller.set_conf('__LeaveStreamsUnattached', '1')
    print(f"Attached listener to EventType.STREAM")

# Resets Tor configuration to default behavior
def reset_tor_config(controller):
    try:
        controller.remove_event_listener(EventType.STREAM)
        print(f"Success: Removed event listener")
    except Exception as e:
        print(f"Warning: could not remove stream listener
- {e}")

    try:
        controller.reset_conf('__LeaveStreamsUnattached')
```

```
        print(f"Success: Reset config")
    except Exception as e:
        print(f"Warning: could not reset config - {e}")

# Sends a GET request through the Tor SOCKS proxy
def send_get_request(url):
    proxies = {
        'http': 'socks5h://127.0.0.1:9050',
        'https': 'socks5h://127.0.0.1:9050',
    }

    try:
        response = requests.get(url, proxies=proxies,
timeout=10)
        print(f"Status Code: {response.status_code}")
        print("Response Snippet:")
        print(response.text[:500])  # print only first 500
characters
        return response.text
    except requests.RequestException as e:
        print(f"Request failed: {e}")
        return None

# Returns all currently built circuits
def get_available_circuits(controller):
    circuits = controller.get_circuits()
    available_circuits = []
    for circ in circuits:
        if circ.status != 'BUILT':
            continue
        available_circuits.append(circ)
    return available_circuits

# Prints a list of all built circuits
def print_available_circuits(controller):
```

```
    print("-" * 20)
    print("Available circuits")
    for i, circ in enumerate(available_circuits):
        print(f"{i+1}. Circuit id: {circ.id} (hops:
{len(circ.path)})")
    print("-" * 20)

# ---- MAIN SCRIPT BEGINS ----

try:
    fetch_latest_consensus()  # Fetch the latest consensus
from Tor directory
finally:
    relays = parse_consensus_file("consensus.txt")  #
Parse the consensus file

# Connect to the Tor control port
with Controller.from_port(port=9051) as controller:
    controller.authenticate()
    print("Connected to Tor!")

    is_quit = False
    try:
        while not is_quit:
            available_circuits =
get_available_circuits(controller)
            if len(available_circuits) == 0:
                # If no circuit exists, prompt user to
create one or exit
                print("-------------------------")
                print("No circuits available")
                print("=========================")
                print(f"1. Create custom circuit")
                print(f"2. Exit")
                choice = int(input("Choose: "))
```

```
                print("--------------------------")
                if choice == 1:
                    hop_count = int(input("Specify
required hop count: "))
                    circuit_id =
create_n_hop_circuit(controller=controller, relays=relays,
hop_count=hop_count)
                    if circuit_id > 0:
                        print(f"Circuit id: {circuit_id}")
                    else:
                        print(f"Failed to create circuit!
Try again...")
                else:
                    is_quit = True
            else:
                # Menu when circuits are available

print("============================================")
                print(f"1. List all available circuit")
                print(f"2. Create custom circuit")
                print(f"3. Attach stream to custom
circuit")
                print(f"4. Reset stream configurations")
                print(f"5. GET request")
                print(f"6. Close tor circuit")
                print(f"7. Exit")
                choice = int(input("Choose: "))
                print("--------------------------")
                if choice == 1:
                    print_available_circuits(controller)

                elif choice == 2:
                    hop_count = int(input("Specify
required hop count: "))
                    print("It may take a while. If it
```

```
takes too long (~15-20s)... please retry.")
                    circuit_id =
create_n_hop_circuit(controller=controller, relays=relays,
hop_count=hop_count)
                    if circuit_id > 0:
                        print(f"Circuit id: {circuit_id}")
                    else:
                        print(f"Failed to create circuit!
Try again...")

                elif choice == 3:
                    print_available_circuits(controller)
                    selected_circ = input("Enter Circuit
ID: ")
                    stream_attached = threading.Event()

attach_stream_to_circuit(controller=controller,
circuit_id=selected_circ)

                elif choice == 4:

reset_tor_config(controller=controller)

                elif choice == 5:
                    url = input("Enter url: ")
                    send_get_request(url)

                elif choice == 6:
                    print_available_circuits(controller)
                    selected_circ = input("Enter Circuit
ID: ")

controller.close_circuit(selected_circ)
                    print(f"Closed circuit
{selected_circ}")
```

```
                else:
                    is_quit = True
    except Exception as e:
        print(f"Error: {e}")

    finally:
        # Always reset config and close controller when
exiting
        reset_tor_config(controller)
        controller.close()
```

Algorithm Explanation

- `fetch_latest_consensus()`
  - This function downloads the most recent Tor relay consensus file from the Tor Project's collector:
    - It fetches the list of available consensus files from the Tor metrics site.
    - Parses the HTML to find all links to consensus files.
    - Picks the latest file based on the sorted list.
    - Downloads the content of the latest consensus file.
    - Saves the file locally as `consensus.txt`.
  - This file contains metadata about all currently active Tor relays and is used later to build custom circuits.
- `weighted_choice(relays)`
  - Purpose: Selects a relay with a probability proportional to its advertised bandwidth.
  - Logic: Calculates total bandwidth, generates a random number within that range, and iterates through the relays

summing bandwidths until the random point is reached.

- `select_guard(relays)`
  - Purpose: Selects a suitable *Guard* relay.
  - Logic: Filters relays with flags `Guard`, `Running`, and `Valid`, then uses `weighted_choice()` to prefer high-bandwidth nodes.
- `select_middle(relays, exclude)`
  - Purpose: Selects a middle relay not in the excluded set.
  - Logic: Filters `Running` and `Valid` relays excluding the specified ones, then uses `weighted_choice()`.
- `select_exit(relays, exclude)`
  - Purpose: Selects an *Exit* relay, ensuring it's not already used in the circuit.
  - Logic: Similar to `select_middle`, but adds the `Exit` flag requirement.
- `select_circuit(relays)`
  - Purpose: Builds a standard 3-hop circuit by selecting one relay for each position (guard, middle, exit).
  - Logic: Sequentially calls the above selection functions and returns the set of selected relays.
- `create_n_hop_circuit(controller, relays, hop_count)`
  - Purpose: Builds a custom-length circuit.
  - Logic: Picks one guard and one exit, then fills the rest with unique middle relays (avoiding repeats). Uses controller to build the circuit.
- `attach_stream_to_circuit(controller, circuit_id)`
  - Purpose: Attaches all new streams to a given circuit ID manually.

    - Logic: Listens for new stream events and binds them to the circuit.
- `get_available_circuits(controller)`
    - Purpose: Lists all currently *BUILT* circuits.
    - Logic: Uses the Tor controller interface to filter circuits by status.